Initiation au développement en C++ : Tripartition, Passage de paramètres et Algorithmique des Tableaux

Thème principal : Bases de la programmation structurée en C++, gestion de la mémoire (valeur vs référence), structures de contrôle et manipulation de tableaux statiques

Synthèse exécutive : Ce cours pose les fondements méthodologiques et techniques indispensables du développement en C++ selon les exigences académiques de l'IUT de Bayonne et du Pays Basque. Il détaille la tripartition universelle d'Etcheverry (Spécification, Déclarations, Traitement), la distinction fondamentale entre le passage de paramètres par valeur et par référence (&), l'utilisation rigoureuse des boucles (bornées et non-bornées), ainsi que la manipulation sécurisée des tableaux statiques et de leurs algorithmes canoniques (tri par sélection et recherche dichotomique). L'objectif est d'acquérir une rigueur de programmation interdisant les effets de bord incontrôlés et les accès mémoire illégaux.

La Tripartition Universelle d'Etcheverry & Spécification

La tripartition est une règle méthodologique stricte imposée pour structurer chaque sous-programme (fonction) en C++. Elle se décompose en trois étapes chronologiques et visuelles dans le code source : 1. **La Spécification** : Placée en commentaire juste au-dessus de l'en-tête de la fonction. Elle doit expliciter le 'But' (ce que fait la fonction), les 'Entrées' (les paramètres nécessaires et leurs contraintes) et les 'Sorties' (la valeur de retour ou les paramètres modifiés). Elle permet à un autre développeur d'utiliser la fonction sans avoir à lire son code. 2. **Les Déclarations** : Situées immédiatement après l'ouverture de l'accolade de la fonction. Toutes les variables intermédiaires nécessaires au traitement doivent y être déclarées et commentées individuellement. Aucune déclaration n'est tolérée plus bas. 3. **Le Traitement** : La partie dynamique de la fonction contenant les instructions, les boucles et les calculs. Cette séparation stricte empêche le code 'spaghetti' et facilite la preuve de correction de l'algorithme.

- Spécification formelle : Définition claire du But, des Entrées (données) et des Sorties (résultats) d'un sous-programme avant d'écrire le code.
- Déclarations locales groupées : Action de déclarer toutes les variables locales en tête de fonction, documentées par leur rôle, pour une lisibilité maximale.
- Traitement algorithmique pur : Séparation stricte de la logique de calcul des phases de déclaration, interdisant l'apparition de variables 'surprises' au milieu du code.

Cas concret enseignant : *M. Etcheverry applique un barème académique très strict : toute fonction mélangeant des déclarations de variables au milieu des instructions de calcul, ou omettant le bloc de spécification (But/Entrées/Sorties), se verra lourdement pénalisée lors des corrections de partiels, même si le programme fonctionne parfaitement à la compilation.*

Passage de Paramètres : Valeur vs Référence (&)

En C++, le choix du mode de passage des paramètres est crucial pour la gestion de la mémoire et la correction des algorithmes : - **Par valeur** : Lorsque vous écrivez `void maFonction(int x)`, le programme duplique la valeur de l'argument lors de l'appel. Si `x` est modifié à l'intérieur de la fonction, cette modification est perdue dès la sortie de la fonction. C'est le comportement par défaut, sûr mais coûteux en mémoire pour les objets volumineux. - **Par référence** : En ajoutant l'esperluette `&` (`void maFonction(int& x)`), la fonction reçoit un alias (l'adresse mémoire directe) de la variable d'origine. Toute modification de `x` dans la fonction altère instantanément et définitivement la variable de l'appelant. C'est indispensable pour renvoyer plusieurs résultats ou modifier un état. - **Par référence constante** : Pour les structures de données lourdes (comme les chaînes de caractères complexes ou les enregistrements), on utilise `const Type&` pour combiner la rapidité du passage par référence (pas de copie en mémoire) et la sécurité du passage par valeur (le compilateur interdit toute modification de la donnée).

• Passage par valeur : Copie de la variable dans la pile d'exécution. La fonction travaille sur une réplique locale sans impacter la variable d'origine.

• Passage par référence (&) : Transmission de l'adresse mémoire de la variable d'origine. Permet de modifier directement la variable de l'appelant.

• Optimisation const Type& : Passage par référence constante pour éviter la copie de structures lourdes tout en interdisant leur modification.

Cas concret enseignant : *Le piège d'examen classique mentionné par l'enseignant concerne la fonction d'échange de deux valeurs : `swap(int a, int b)`. Si vous oubliez les symboles `&` dans l'en-tête (`swap(int& a, int& b)`), les variables s'échangeront correctement dans l'espace mémoire local de la fonction, mais resteront désespérément inchangées dans la fonction principale `main()` après l'appel.*

Structures Répétitives & Gestion de Sentinelle

Le choix d'une structure de contrôle répétitive doit répondre à une logique algorithmique précise : - **La boucle `for`** est réservée aux parcours d'intervalles ou de structures indexées (comme les tableaux) dont on connaît la taille exacte à l'avance. - **La boucle `while`** évalue sa condition d'entrée *avant* d'exécuter son bloc. Si la condition est fausse dès le départ, le bloc n'est jamais exécuté. Elle est parfaite pour traiter des flux de données s'arrêtant sur une 'valeur sentinelle' (par exemple, lire des notes jusqu'à ce que l'utilisateur saisisse `-1`). - **La boucle `do...while`** évalue sa condition *après* l'exécution du bloc. Elle garantit ainsi que les instructions internes seront exécutées au moins une fois. C'est la structure standard pour forcer un utilisateur à saisir une valeur valide (par exemple, un entier compris entre 1 et 10).

• Boucle bornée (for) : Utilisée lorsque le nombre d'itérations est connu et déterminé avant l'entrée dans la boucle.

• Boucle non-bornée à pré-condition (while) : Utilisée lorsque l'arrêt dépend d'un événement ou d'une valeur sentinelle, avec un test avant chaque exécution.

• Boucle non-bornée à post-condition (do-while) : Garantit au moins une exécution du bloc d'instructions, idéale pour le contrôle de saisie utilisateur.

Cas concret enseignant : *L'enseignant insiste sur une règle de programmation propre : l'interdiction formelle d'utiliser des boucles infinies artificielles du type `while(true)` combinées à des instructions de rupture brutale comme `break` au milieu du bloc. La condition d'arrêt doit toujours être explicitée et maîtrisée directement dans l'en-tête de la boucle `while` pour préserver la clarté logique.*

Tableaux Statiques & Algorithmes Canoniques

Un tableau statique en C++ est déclaré avec une taille fixe connue dès la compilation (ex: `int tab[5]`). Les éléments sont accessibles via des indices de type entier non-signé (`unsigned int`) pour éviter les indices négatifs incohérents. L'indice du premier élément est toujours `0`, et celui du dernier est `taille - 1`. Deux algorithmes fondamentaux doivent être maîtrisés : 1. **Le tri par sélection** : On parcourt le tableau pour trouver le plus petit élément, puis on l'échange avec l'élément à l'indice courant. On répète l'opération pour chaque position du tableau. Sa complexité est de $O(n^2)$, ce qui le rend inefficace sur de très grands volumes de données mais simple à implémenter. 2. **La recherche dichotomique** : Au lieu de parcourir le tableau élément par élément (recherche séquentielle en $O(n)$), on divise l'espace de recherche par deux à chaque étape en comparant l'élément central avec la valeur recherchée. Sa complexité est de $O(\log n)$, mais elle exige que le tableau soit préalablement trié.

• Tableau statique : Structure de données de taille fixe, stockée de manière contiguë en mémoire, indexée de 0 à TAILLE - 1.

• Tri par sélection : Algorithme de tri de complexité quadratique O(n²) consistant à rechercher le minimum successif pour le placer au début.

• Recherche dichotomique : Algorithme de recherche ultra-rapide de complexité logarithmique O(log n), nécessitant impérativement un tableau trié.

Cas concret enseignant : *Attention au dépassement de bornes ('Out of Bounds') ! Si vous tentez d'accéder à `tab[n]` pour un tableau de taille `n`, le compilateur C++ ne générera pas d'erreur à la compilation. En revanche, à l'exécution, cela provoquera un comportement indéfini : soit un plantage immédiat de type 'Segmentation Fault', soit une corruption silencieuse de la mémoire de la pile, modifiant d'autres variables à votre insu.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne peut-on pas retourner deux valeurs différentes avec l'instruction 'return' dans une fonction C++ ?
Réponse du professeur : Par conception, l'instruction `return` en C++ ne peut renvoyer qu'un seul type de donnée (une seule valeur ou un seul objet). Pour contourner cette limitation de manière propre et académique, on utilise le passage de paramètres par référence (`&`). On passe à la fonction des variables 'réceptacles' qui seront modifiées directement à l'intérieur de la fonction. Une autre solution consiste à regrouper ces valeurs dans un enregistrement personnalisé (`struct`).

Question étudiant : Quelle est la condition obligatoire pour pouvoir appliquer une recherche dichotomique sur un tableau ?
Réponse du professeur : Le tableau doit être impérativement trié au préalable (par ordre croissant ou décroissant). Si le tableau n'est pas trié, l'algorithme de dichotomie éliminera des pans entiers du tableau contenant potentiellement la valeur recherchée, rendant le résultat totalement faux.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C++ respectant scrupuleusement la tripartition d'Etcheverry permettant de calculer la moyenne des éléments strictement positifs d'un tableau statique d'entiers. Si le tableau ne contient aucun élément positif, la fonction devra renvoyer la valeur -1.0 pour signaler cette impossibilité et éviter une division par zéro.

Résolution pas à pas :
1. Rédiger la spécification complète (But, Entrées, Sorties). 2. Déclarer l'en-tête de la fonction en utilisant un tableau d'entiers constants (pour ne pas le modifier) et un entier non-signé pour sa taille. 3. Déclarer toutes les variables locales au début du bloc (somme, compteur d'éléments positifs, indice de boucle). 4. Parcourir le tableau à l'aide d'une boucle 'for' bornée. 5. Tester chaque élément : s'il est strictement supérieur à 0, l'ajouter à la somme cumulée et incrémenter le compteur. 6. Après la boucle, vérifier si le compteur est égal à 0 pour éviter la division par zéro. Si oui, retourner -1.0. 7. Effectuer une conversion de type explicite (static_cast) de la somme pour garantir une division flottante précise lors du calcul de la moyenne.

Résultat : ```cpp // ========== // BUT : Calculer la moyenne des éléments strictement positifs d'un tableau // ENTRÉES : // - tab : le tableau d'entiers à analyser (constant) // - nbElts : le nombre d'éléments du tableau (entier non-signé) // SORTIE : // - La moyenne des valeurs positives (double), ou -1.0 si aucune valeur positive // ========== double calculerMoyennePositifs(const int tab[], unsigned int nbElts) { // ---------- // DÉCLARATIONS LOCALES // ---------- int somme = 0; // Accumulateur pour la somme des positifs unsigned int compteur = 0; // Nombre d'éléments positifs trouvés unsigned int i; // Indice de boucle de parcours // ---------- // TRAITEMENT // ---------- for (i = 0; i < nbElts; ++i) { if (tab[i] > 0) { somme += tab[i]; compteur++; } } // Sécurité pour éviter la division par zéro if (compteur == 0) { return -1.0; } // Cast explicite pour obtenir une division réelle (double) return static_cast(somme) / compteur; } ```

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre l'utilisation de 'cin >> val' et 'getline(cin, chaine)' pour lire une saisie utilisateur ?

R : 'cin >> val' s'arrête de lire dès qu'il rencontre un caractère d'espacement (espace, tabulation, retour à la ligne), ce qui le rend inadapté pour saisir des phrases contenant des espaces. À l'inverse, 'getline(cin, chaine)' lit toute la ligne saisie par l'utilisateur, espaces compris, jusqu'à ce qu'il appuie sur Entrée.

Q : Quel mot-clé du langage C++ garantit qu'une variable ou un paramètre ne pourra jamais être modifié après son initialisation ?

R : Le mot-clé 'const'.

Q : Dans quelle phase de la tripartition d'Etcheverry doit-on déclarer les variables locales d'une fonction ?

R : Dans la deuxième phase (Déclarations), qui se situe obligatoirement en tête de la fonction, juste après l'accolade ouvrante et avant toute instruction de traitement.